Local Personal Cards for Cross-Session Preference Transfer to Frozen Language Models

Anonymous authors
Paper under double-blind review

Abstract

A user's chat history contains useful preference signals, but sending that history to a remote language model exposes more personal data than the model needs. We study a local alternative. A Qwen3-0.6B model with a LoRA adapter reads past tasks, assistant answers, and explicit user corrections on the user's machine. It emits schema-validated Personal Cards that record task type, feedback facets, preference direction, confidence, scope, and a short instruction. A deterministic profile builder transfers a rule only when the same instruction is supported by at least two distinct records. A frozen Sonnet 4.5 model then receives the current task and this compact profile, but not the earlier conversations or card evidence text.

We train the local extractor on 914 examples from one user's archive. On 183 conversation-disjoint test examples, the adapter produces valid card JSON for every input, with 0.705 task-type accuracy and 0.732 feedback micro F1. The untouched base model produces no valid card under the same prompt and decoding protocol; this is a structural control rather than a strong baseline. A single-card transfer diagnostic shows no advantage. With repeated support, the profile wins 5 of 7 position-consistent comparisons on eight new tasks. Mean task quality is unchanged at 4.25, while mean preference fit rises from 3.375 to 4.0. However, paired bootstrap intervals for both deltas include zero. These results show that the full local-extraction and remote-transfer loop is feasible, not that it yields a reliable personalization gain. The study covers one user, uses the same model family as executor and judge, and offers data minimization rather than a formal privacy guarantee.

1 Introduction

A correction such as "give me a table, not another long explanation" can be useful beyond the conversation in which it appeared. The obvious solution is to retrieve the earlier exchange and place it in the next model prompt. That also sends the old task, answer, and feedback to the remote provider.

We ask a narrower question: can a small local model turn past corrections into a compact profile that helps a frozen remote model on later tasks? The local model may read the sensitive history. The remote model may receive the current task and a short derived profile, but not the old conversations.

Our system has three steps. First, deterministic rules build supervised examples from one user's archive. Second, a locally trained Qwen3-0.6B extractor maps each observation to a structured Personal Card. Third, a local SQLite store groups cards by exact instruction and transfers only repeatedly supported rules. The remote Sonnet 4.5 executor is never fine-tuned. It is guided through its normal system-prompt interface.

This split is useful for closed models. Personalized fine-tuning changes the model that produces the answer (Li et al., 2024); soft-prompt and decoding-time methods need embedding or logit access (Hebert et al., 2024; Lv et al., 2025). Our executor exposes neither. A read-

able text profile works with the same interface used by a command-line agent or hosted assistant.

The evidence is deliberately small. The archive represents one user, and the online comparison has eight tasks. We therefore report the negative single-card run, the positive point estimate from repeated support, both zero-crossing bootstrap intervals, and concrete failures. Our contributions are:

- a fixed Personal Card schema and three-way evidence contract for explicit corrections, verified revisions, and category-only abstentions;
- a local 0.6B extractor trained on 914 archive examples, with conversation-disjoint evaluation and a complete H100 training record;
- a repeated-support profile builder that blocks one-off rules and exposes its ranking and transfer metadata;
- an order-reversed cross-session evaluation with task quality, preference fit, uncertainty, latency, cost, and failure analysis.

2 Related Work

Preferences from feedback and edits. P-RLHF learns personalized language or reward models from user-specific feedback (Li et al., 2024). CIPHER is closer to our data source: it infers preferences from user edits and reuses them for later contexts (Gao et al., 2024). Our remote executor remains frozen, while a separate small model extracts a discrete local record. The repeated-support rule also blocks a correction that appears only once.

Personalization at the model interface. PERSOMA compresses history into learned soft prompts (Hebert et al., 2024). CoSteer changes a cloud model's token distribution during decoding with a local delta model (Lv et al., 2025). Both are more expressive than plain text, but they need access that a prompt-only API does not expose. Personal Cards use ordinary text and can be inspected or deleted before use. LaMP studies retrieval-based personalization across users and tasks (Salemi et al., 2023). General memory systems such as MemGPT and Mem0 also carry information across sessions (Packer et al., 2023; Chhikara et al., 2025). Our focus is narrower: reduce the old conversation to a short preference artifact before the remote call.

Evaluation. LLM judges show position bias, so we judge each answer pair in both orders (Zheng et al., 2023). They can also favor generations from their own model family (Panickssery et al., 2024). We cannot remove that risk because Sonnet 4.5 is both executor and judge, but we state it and report every order-inconsistent pair. We use paired bootstrap intervals to show the large uncertainty from eight tasks (Efron, 1979).

3 Method

3.1 Local and Remote Roles

Let an historical observation be

$$o = (x, a, f, r),$$

where x is the earlier task, a is the assistant answer, f is the user's feedback, and r records whether a revised answer passed a local deterministic check. The local extractor E_θ returns a Personal Card $c = E_\theta(o)$. The remote executor R receives a new task x' and, in the guided condition, a rendered profile P_τ for task type τ:

$$y_{\text{base}} = R(x'), \qquad y_{\text{profile}} = R(P_\tau, x').$$

The remote executor never receives o or the card's evidence sentence.

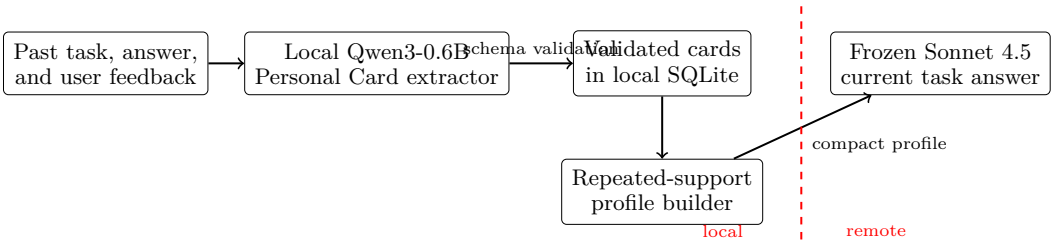

Figure 1: The old conversation and card evidence stay local. The remote model receives the current task and a compact profile derived from eligible cards.

3.2 Personal Card Schema

Each card is a JSON object with ten required fields:

- task_type: one of eight archive task classes;
- feedback_types: a nonempty subset of fifteen facets;
- direction: like, dislike, or uncertain;
- likert: an integer from 1 to 5;
- confidence: a value in $[0, 1]$;
- scope: global, task_type, or task_instance;
- instruction: a reusable instruction of at most 240 characters;
- evidence: a short local explanation of at most 160 characters;
- evidence_type: the source class described below; and
- privacy: the constant local_only.

The extractor emits the card only. It does not answer the task.

The evidence contract has three classes. An explicit_correction is a real corrective user message. A verified_revision is an assistant revision that passes a deterministic check for the requested change; it is weaker than user approval. A category_label has no explicit preference evidence. Because every schema field is required, an abstention card uses no_explicit_feedback, direction uncertain, Likert 3, confidence 0.4, scope task_instance, and an instruction that says not to infer a lasting preference. Such cards train abstention and are never eligible for transfer.

3.3 Offline Training

The archive builder creates targets with deterministic local rules. Explicit corrections produce negative examples. A revision is added as a weak positive only when it passes the requested-change check. Category-only examples teach task classification without a preference claim. Conversation IDs, rather than rows, are assigned to train, validation, and test.

We freeze the Qwen3-0.6B backbone (Yang et al., 2025) and train LoRA modules (Hu et al., 2022) on q_proj, k_proj, v_proj, and o_proj. Only target JSON tokens contribute to the loss:

$$\mathcal{L}(\theta) = -\frac{1}{|T_c|} \sum_{t \in T_c} \log p_\theta(y_t \mid y_{<t}, o), \tag{1}$$

where T_c indexes the card completion. Prompt and observation tokens remain visible through attention but receive label -100 and do not enter the loss.

Table 1: Offline dataset. Evidence counts and split counts describe different partitions of the same 914 examples.

Item	Count
Explicit corrections	260
Verified revisions	39
Category-only abstentions	615
Train	582
Validation	149
Test	183

3.4 Repeated-Support Profile

The profile store first rejects category-only cards, cards with direction uncertain, confidence below 0.65, and cards with task_instance scope. Each retained row stores a hash of the conversation ID and the full observation. The hash identifies a source record; records from the same conversation can still have different hashes.

For an exact instruction string u, let $H(u)$ be its distinct evidence hashes. The support is

$$\text{supp}(u) = |H(u)|.$$

The instruction is transferable only when

$$\text{supp}(u) \geq 2. \tag{2}$$

The store ranks candidates by

$$s(u) = \sum_{i:u_i=u} \gamma_i w(e_i), \qquad w(e_i) = \begin{cases} 1, & e_i = \text{explicit\_correction}, \\ 0.5, & e_i = \text{verified\_revision}, \end{cases} \tag{3}$$

where duplicate hashes contribute once. It then keeps at most three rules, skipping a lower-ranked rule when it adds no feedback facet.

The rendered profile is a small JSON block. It contains the task type; each selected instruction; its feedback facets, support count, mean confidence, mean Likert value, and scope; and a policy stating that the current request overrides the profile. It contains no raw conversation, card evidence sentence, source hash, or conversation identifier. Thus the method minimizes what is sent, but the derived profile is still personal information.

4 Experimental Design

4.1 Offline Data

The dataset contains 914 examples from one user's archive. Table 1 shows both evidence types and conversation-disjoint splits.

4.2 Card Extraction

The LoRA adapter and untouched base model receive the same extraction prompt, test examples, greedy decoding settings, and validator. We report valid-card rate, task-type accuracy, direction accuracy, exact Likert accuracy, scope accuracy, exact feedback-facet set accuracy, and pooled feedback micro F1. An invalid output scores zero on all field metrics. The untouched model is a format-and-task control, not a competitive instruction-tuned baseline.

4.3 Cross-Session Transfer

The first diagnostic applies one eligible card to one new task of the same type. Its eight task IDs are then excluded from the main run.

Table 2: Held-out Personal Card extraction on 183 examples.

Metric	Base	LoRA
Valid card JSON	0.000	1.000
Task-type accuracy	0.000	0.705
Direction accuracy	0.000	1.000
Likert exact accuracy	0.000	0.831
Scope accuracy	0.000	0.995
Feedback exact set	0.000	0.639
Feedback micro F1	0.000	0.732

For the main run, the local store is populated from the adapter predictions. It holds 74 candidate rows after the per-card source, direction, confidence, and scope filters. The case selector chooses eight new tasks from the separate holdout file before the first executor call. It rejects tasks that depend on missing attachments or surrounding context and alternates across task types with available profiles.

For each task, Sonnet 4.5 answers once with the base system prompt and once with the rendered profile appended to that prompt. The same model family judges the pair twice with answer order reversed. The judge sees the current task, the compact profile, and both candidate answers. It returns a winner and 1–5 scores for task quality and preference fit. A pair is position-consistent only when both orderings select the same underlying answer.

We average the two judge orderings for each condition and compute paired deltas. To show uncertainty, we resample the eight paired tasks 20,000 times with seed 20260905 and report percentile 95% intervals.

5 Results

5.1 H100 Training and Card Extraction

Training uses four epochs, micro-batches of 2, gradient accumulation of 8, rank 16, alpha 32, dropout 0, and learning rate 2×10^{-4}. The frozen backbone has 4,587,520 trainable LoRA parameters. The raw training split has 582 examples. Repeating the 20 verified revisions in that split five times produces 662 loader examples per epoch.

The run uses one NVIDIA H100 80GB HBM3. Training inside the process takes 260.76 seconds and reaches validation loss 0.0667. Peak allocated GPU memory is 25.53 GiB. No training or validation example is truncated at the 4,096-token limit.

Table 2 gives held-out results. The adapter produces a valid card for all 183 examples. Direction and evidence type are exact on every example, while task-type and feedback-facet prediction remain the main error sources. The untouched base model produces no valid card under this protocol.

5.2 Single Cards Do Not Transfer Reliably

The single-card diagnostic produces three guided wins, three baseline wins, one tie, and one position-inconsistent pair. This gives no net advantage. Manual inspection shows why: one correction can encode a file name, number, or other detail that should not move to another task. This result motivated the support threshold in Eq. equation 2.

5.3 Repeated Support

Table 3 reports the main run. The profile wins five of seven position-consistent comparisons. Mean task quality is identical in the two conditions. Mean preference fit is 0.625 points higher with the profile.

Table 3: Repeated-support evaluation on eight new tasks. Deltas are profile minus baseline.

Measure	Result
Profile / baseline / inconsistent	5 / 2 / 1
Profile win rate among consistent pairs	0.714
Mean task quality, baseline / profile	4.250 / 4.250
Mean preference fit, baseline / profile	3.375 / 4.000
Paired task-quality delta	0.000
Paired preference-fit delta	+0.625
95% interval, task-quality delta	[−0.6875, 0.5625]
95% interval, preference-fit delta	[−0.25, 1.5]
Mean latency, baseline / profile	21.32 s / 26.24 s
Total executor and judge cost	$1.923

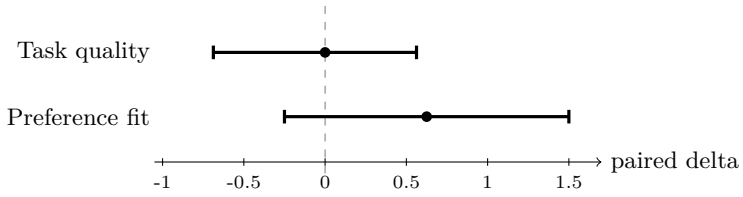

Figure 2: Paired point estimates and 20,000-resample percentile intervals. Both intervals include zero.

The intervals in Figure 2 are wide and include zero. The point estimates are consistent with improved preference fit without lower mean task quality, but they do not establish a reliable effect. We report this as feasibility evidence.

The profile adds 4.93 seconds of mean executor latency in this run. The complete main evaluation uses 16 executor calls and 16 judge calls and costs $1.923. These measurements describe this task set and profile length; they do not identify the cause of the latency difference.

5.4 Failure Analysis

The manual audit identifies clear profile wins on repository upload instructions, a short translation, campaign questions, and a Newton convergence explanation. It also finds three different failure modes.

First, the guided least-squares answer states that its displayed data sum to 54, although they sum to 53. More detailed structure did not protect against arithmetic error. Second, both answers to the degrees-of-freedom question are too broad and can leave a false impression about known variance. The profile cannot repair an error shared by the executor. Third, the foreign-key pair changes winner when answer order is reversed. That pair is reported as position-inconsistent rather than forced into a win or loss.

We also tried a Qwen2.5-7B-Instruct teacher that rewrote individual corrections as broader rules. We cancelled the branch after 216 labels: 201 had task_instance scope, and 9 were deterministic fallbacks after invalid model output. No student model was trained from these labels. This is a failed engineering branch, not an ablation result.

6 Discussion and Limitations

The offline result is the strongest part of the study. A 0.6B local model can learn the card schema and abstention behavior in a few GPU minutes. Its task classification accuracy, however, is only 0.705. Since task type routes a card to a profile bucket, these errors can

split support across the wrong buckets. The 0.639 exact feedback-set accuracy can also change which rules survive facet deduplication.

Repeated support is useful because it provides a clear transfer gate. It does not solve semantic equivalence: two paraphrases count as different instructions. The failed 7B branch shows that learned rewriting can over-generalize or copy one-off details, but our current exact-string rule can under-generalize. A future method should cluster paraphrases while preserving source-level auditability.

The online stage changes local memory, not model weights. It therefore supports fast profile updates but does not test online fine-tuning. The benchmark also has one user and eight tasks. Sonnet 4.5 acts as executor and judge, and the judge sees the same profile used to guide one answer. Preference fit therefore measures compliance with the displayed profile, not independent user satisfaction. Human judgments and a cross-family judge are needed before making a stronger claim.

Finally, "local" does not mean private in a formal sense. Raw history and evidence text stay on the user's machine, but support counts, confidence, Likert summaries, facets, and derived instructions are sent to the remote model. We provide no differential-privacy or cryptographic guarantee (Dwork et al., 2006). The source archive is also private and cannot be released without review and consent, which limits exact external reproduction. Persistent profiles require clear user controls for inspection and deletion before deployment.

7 Conclusion

We built and tested a complete path from one user's historical corrections to a local Personal Card store and then to a frozen remote model. The local extractor is cheap and reliably emits valid structured cards. Single-card transfer fails, while repeated support gives a positive but uncertain signal on eight new tasks. The result is a working, auditable feasibility prototype. The next evidence should come from more users and tasks, stronger extractor baselines, and independent judges rather than from stronger wording around the current numbers.

A Exact Data and Schema Details

A.1 Task and Feedback Vocabularies

The eight task values in the implementation are knowledge explanation, proof/problem solving, code development, operations/troubleshooting, format processing, writing/editing, research analysis, and planning. The stored enum values are the corresponding Chinese labels used by the source archive.

The fifteen feedback facets are: verbosity, detail_depth, format_structure, tone_style, language, correctness, constraint_adherence, context_memory, clarification, tool_use, workflow_compliance, code_runnability, source_verification, task_success, and no_explicit_feedback.

A.2 Evidence Counts by Split

Table 4: Evidence type within each conversation-disjoint split.

Evidence	Train	Validation	Test	Total
Explicit correction	146	51	63	260
Verified revision	20	7	12	39
Category label	416	91	108	615
Total	582	149	183	914

A.3 Validation

The Python validator checks the task and feedback enums, direction, Likert range, confidence range, scope, nonempty instruction and evidence strings, their length limits, evidence type, and the local_only privacy constant. The prediction parser accepts a JSON object found in the completion and then applies this validator. Invalid cards are recorded as failed predictions and never enter the profile store.

B Training Configuration

Table 5: Offline SFT configuration.

Setting	Value
Backbone	Qwen3-0.6B, frozen
LoRA targets	q_proj, k_proj, v_proj, o_proj
Rank / alpha / dropout	16 / 32 / 0
Trainable parameters	4,587,520
Epochs	4
Learning rate	2×10^{-4}
Micro-batch / accumulation	2 / 8
Optimizer	AdamW, weight decay 0.01
Schedule	cosine, 6% warmup
Precision	bfloat16
Maximum sequence length	4,096
Verified-revision repetition	5
Random seed	17

C Per-Task Online Results

Table 6: Main online run. Score deltas are profile minus baseline after averaging the two judge orderings.

Task	Verdict	Δ quality	Δ preference
Booth naming	baseline	−0.5	−0.5
Repository upload	profile	+1.0	+0.5
Degrees of freedom	profile (weak)	+1.0	0.0
Short translation	profile	+0.5	+2.0
Least squares	baseline	−2.0	−1.5
Campaign questions	profile	−0.5	+2.5
Newton convergence	profile	+0.5	+1.5
Foreign-key guidance	inconsistent	0.0	+0.5

The manual audit does not replace a blind human study. It labels the repository, translation, campaign, and Newton cases as clear profile wins. The degrees-of-freedom win is weak because both answers are potentially misleading. Booth naming and least squares are clear baseline wins. The foreign-key pair remains unresolved.

D Artifact Map and Next Experiments

The next experiments are intentionally left open rather than reported as results:

1. repeat both tracks on another consenting user's archive;
2. judge the stored answer pairs with blinded humans and another model family;

Table 7: Authoritative local artifacts.

Artifact	Content
data_balanced_v2/dataset_manifest.json	dataset size, source counts, split counts, and disjointness flag
outputs/qwen3_0p6b_personal_card_balanced_v2/	adapter, training summary, base and adapter predictions, extraction metrics
outputs/online_sonnet45_balanced_v2/	single-card diagnostic protocol, answer pairs, and summary
outputs/online_sonnet45_repeated_profile_v2/	main protocol, SQLite profile, answer pairs, paired metrics, and manual audit
outputs/archive_local_teacher_failed_scope_20260905/	cancelled 7B teacher branch and failure record

3. compare the LoRA extractor with an instruction-tuned zero-shot extractor under the same schema and metrics;
4. freeze a larger set of new tasks before generation and rerun the paired protocol;
5. test semantic grouping of paraphrased rules while retaining distinct source support and an auditable transfer log.

References

P. Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. pp. 2993–3000, 2025.

C. Dwork, Frank McSherry, Kobbi Nissim, and Adam D. Smith. Calibrating noise to sensitivity in private data analysis. J. Priv. Confidentiality, 7:17–51, 2006.

B. Efron. Bootstrap methods: Another look at the jackknife. Annals of Statistics, 7:1–26, 1979.

Ge Gao, Alexey Taymanov, Eduardo Salinas, Paul Mineiro, and Dipendra Misra. Aligning llm agents by learning latent preference from user edits. arXiv preprint arXiv:2404.15269, 2024.

Liam Hebert, Krishna Sayana, Ambarish Jash, Alexandros Karatzoglou, Sukhdeep Sodhi, Sumanth Doddapaneni, Yanli Cai, and Dima Kuzmin. PERSOMA: PERsonalized SOft proMpt Adapter architecture for personalized language prompting. arXiv preprint arXiv:2408.00960, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations, 2022.

Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Liu Leqi. Personalized language modeling from personalized human feedback. arXiv preprint arXiv:2402.05133, 2024.

Hang Lv, Sheng Liang, Hao Wang, Hongchao Gu, Yaxiong Wu, Wei Guo, Defu Lian, Yong Liu, and Enhong Chen. CoSteer: Collaborative decoding-time personalization via local delta steering. arXiv preprint arXiv:2507.04756, 2025.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph Gonzalez. Memgpt: Towards llms as operating systems. ArXiv, abs/2310.08560, 2023.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. Llm evaluators recognize and favor their own generations. ArXiv, abs/2404.13076, 2024.

Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. Lamp: When large language models meet personalization. ArXiv, abs/2304.11406, 2023.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxin Yang, Jingren Zhou, Jingren Zhou, Junyan Lin, K. Dang, Keqin Bao, Ke-Pei Yang, Le Yu, Li-Chun Deng, Mei Li, Min Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yi-Chao Zhang, Ying-Er Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. 2025.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, E. Xing, Haotong Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. ArXiv, abs/2306.05685, 2023.